

Bias and Variance

Date: March 3, 2026

So far in this class, we have seen many different kinds of machine learning models that often differ greatly in their model complexity. We have seen both linear models—linear regression, logistic regression, and so on—as well as nonlinear models—decision trees, kernel methods, and more. Even more, many of these learning algorithms offer even more fine-grained control over model complexity than simply choosing the class of models via *hyperparameters*: you might choose the degree of a polynomial kernel; the regularization parameter λ in a linear model; the depth of a decision tree.

Up to this point, the solution to these problems has been cross validation: you first estimate the generalization error as you vary one of the above parameters (say, a regularization parameter) on a validation set, and choose the best one. The problem is that, as you add more tools to your machine learning toolkit, it can become pretty difficult to simply run cross validation over all possible models you might choose and all possible hyperparameters.

In this lecture, we are going to try and do a better job of understanding the error of a machine learning algorithm in general. Our goal will be to break down the generalization error into components that we can understand, and from these components derive very good rules of thumb that will let you decide whether to use a more or less complex model without having to actually train all of them! It's this last ability that makes this material so critical: mastering the material here will enable you to try a machine learning model, and then immediately have a pretty good idea of what to try next instead of simply having to try everything.

1 Bias-Variance Decomposition

There are a number of ways to understand generalization error. In this lecture, we are going to focus on one in particular: the *Bias-Variance decomposition*. This decomposition will let us decompose the (expected) generalization error of a machine learning algorithm $\mathcal{A}$ into three components: one corresponding to **bias** (technically bias^2), one corresponding to **variance**, and one corresponding to **noise**. Equipped with this decomposition, we will then focus on two questions:

1. How do you detect in practice whether you are in a setting where your generalization error is dominated by bias or by variance?
2. Once you know which kind of error is dominating, what should you do about it?

1.1 Problem Setup

To begin with, we will assume labeled instances (x, y) are drawn from some distribution, $(\mathbf{x}, y) \sim \mathcal{P}(\mathbf{x}, y)$. Here, we assume the regression setting ($y \in \mathbb{R}$) with access to d features $\mathbf{x} \in \mathbb{R}^d$. D

will refer to a labeled dataset of n training examples drawn i.i.d. from this distribution: $D : \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\} \stackrel{i.i.d.}{\sim} \mathcal{P}^n$.

Using a particular dataset D , we will use some machine learning algorithm $h_D \leftarrow \mathcal{A}(D)$ to train a model $h_D : \mathbb{R}^d \rightarrow \mathbb{R}$, with the informal goal of producing $h_D(\mathbf{x}) \approx y$ (e.g., h_D should predict the label y from an input $\mathbf{x}$).

Optimal regressor $h^*(\mathbf{x})$. In order to talk about the error of a regressor and pin down a specific “optimal” regressor, it’s important that we first pin down a specific notion of error. As you’ve seen in the past, the optimal regressor depends on the specific notion of generalization error we choose to use. Throughout what follows, we will restrict our attention to the squared error for the regression setting:

$$\text{er}^{\text{sq}}[h_D] = \mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{P}} \left[(y - h_D(\mathbf{x}))^2 \right]$$

Under this choice of error, we know from factoring $\mathcal{P}(\mathbf{x}, y) = p(y | \mathbf{x})p(\mathbf{x})$ that the optimal regression model (Bayes optimal) would simply be the expected label given $\mathbf{x}$ (recall Homework 2):

$$h^*(\mathbf{x}) = \mathbb{E}_{y|\mathbf{x}}[y] = \int y p(y | \mathbf{x}) dy$$

Expected error of a regressor h_D . For a specific model h_D returned to us from a machine learning algorithm run on D (e.g., $h_D = \mathcal{A}(D)$), the expected error of the regressor is given by:

$$\mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{P}} \left[(h_D(\mathbf{x}) - y)^2 \right]$$

This measures how well a *particular* model h_D does, averaged over all possible datapoints drawn from $\mathcal{P}(\mathbf{x}, y)$.

Expected regressor of an algorithm $\bar{h}$. The above error analyzes the expected generalization error of a specific model h_D returned from a machine learning algorithm $\mathcal{A}$. What we would like to do instead is to somehow analyze the properties of $\mathcal{A}$ directly. One way to do this would be to consider the model $\bar{h}$ that we would get if we *averaged* the results of $\mathcal{A}$ over all possible datasets we could draw $D \sim \mathcal{P}^n$:

$$\bar{h}(\mathbf{x}) = \mathbb{E}_{D \sim \mathcal{P}^n} [h_D(\mathbf{x})]$$

$\bar{h}(\mathbf{x})$ is the model that first trains the model $h_D(\mathbf{x})$ on every possible dataset D , and then takes the weighted average of all of these models’ predictions. In some ways, this captures the “average behavior” of the entire machine learning algorithm $\mathcal{A}$, especially if we treat the dataset D as a random variable where we don’t know what dataset we are going to be faced with in advance.

1.2 Decomposing the expected error of an algorithm $\mathcal{A}$

With the above, we can now finally define the quantity we want to examine further. In particular, if we treat D as a random variable, either because we don't know what particular dataset we are going to be faced with in advance, or simply because we would like to reason about their "average" performance of an algorithm $\mathcal{A}$ on data drawn from a distribution $\mathcal{P}(\mathbf{x}, y)$, then we can define the expected error

$$\mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{P}} \left[\mathbb{E}_{D \sim \mathcal{P}^n} \left[(h_D(\mathbf{x}) - y)^2 \right] \right]$$

Throughout, we will shorthand the double expectation above as $\mathbb{E}_{\mathbf{x}, y, D}[\cdot]$.

This is the expected error of a model h_D returned by a machine learning algorithm $\mathcal{A}$, averaged over all possible test points and all possible datasets that we could have trained h_D on. To decompose this, we first both subtract and add $\bar{h}(\mathbf{x})$ from the inner expectation and expand:

$$\begin{aligned} \mathbb{E}_{\mathbf{x}, y, D} \left[(h_D(\mathbf{x}) - y)^2 \right] &= \mathbb{E}_{\mathbf{x}, y, D} \left[[(h_D(\mathbf{x}) - \bar{h}(\mathbf{x})) + (\bar{h}(\mathbf{x}) - y)]^2 \right] \\ &= \mathbb{E}_{\mathbf{x}, D} \left[(h_D(\mathbf{x}) - \bar{h}(\mathbf{x}))^2 \right] + \mathbb{E}_{\mathbf{x}, y} \left[(\bar{h}(\mathbf{x}) - y)^2 \right] + 2 \mathbb{E}_{\mathbf{x}, y, D} \left[(h_D(\mathbf{x}) - \bar{h}(\mathbf{x})) (\bar{h}(\mathbf{x}) - y) \right] \end{aligned}$$

The last of these three terms is zero:

$$\begin{aligned} \mathbb{E}_{\mathbf{x}, y, D} \left[(h_D(\mathbf{x}) - \bar{h}(\mathbf{x})) (\bar{h}(\mathbf{x}) - y) \right] &= \mathbb{E}_{\mathbf{x}, y} \left[\left(\mathbb{E}_D [h_D(\mathbf{x}) - \bar{h}(\mathbf{x})] \right) (\bar{h}(\mathbf{x}) - y) \right] \\ &= \mathbb{E}_{\mathbf{x}, y} \left[\left(\mathbb{E}_D [h_D(\mathbf{x})] - \bar{h}(\mathbf{x}) \right) (\bar{h}(\mathbf{x}) - y) \right] \\ &= \mathbb{E}_{\mathbf{x}, y} \left[(\bar{h}(\mathbf{x}) - \bar{h}(\mathbf{x})) (\bar{h}(\mathbf{x}) - y) \right] \\ &= \mathbb{E}_{\mathbf{x}, y} \left[(0) (\bar{h}(\mathbf{x}) - y) \right] \\ &= 0 \end{aligned}$$

Thus, returning to the previous expression, we have:

$$\mathbb{E}_{\mathbf{x}, y, D} \left[(h_D(\mathbf{x}) - y)^2 \right] = \underbrace{\mathbb{E}_{\mathbf{x}, D} \left[(h_D(\mathbf{x}) - \bar{h}(\mathbf{x}))^2 \right]}_{\text{Variance}} + \mathbb{E}_{\mathbf{x}, y} \left[(\bar{h}(\mathbf{x}) - y)^2 \right]$$

Breaking down $\mathbb{E}_{\mathbf{x}, y} \left[(\bar{h}(\mathbf{x}) - y)^2 \right]$. We can break down the second of these terms as follows by subtracting and adding $h^*(\mathbf{x})$:

$$\begin{aligned} \mathbb{E}_{\mathbf{x}, y} \left[(\bar{h}(\mathbf{x}) - y)^2 \right] &= \mathbb{E}_{\mathbf{x}, y} \left[[(\bar{h}(\mathbf{x}) - h^*(\mathbf{x})) + (h^*(\mathbf{x}) - y)]^2 \right] \\ &= \underbrace{\mathbb{E}_{\mathbf{x}, y} \left[(h^*(\mathbf{x}) - y)^2 \right]}_{\text{Noise}} + \underbrace{\mathbb{E}_{\mathbf{x}} \left[(\bar{h}(\mathbf{x}) - h^*(\mathbf{x}))^2 \right]}_{\text{Bias}^2} + 2 \mathbb{E}_{\mathbf{x}, y} \left[(\bar{h}(\mathbf{x}) - h^*(\mathbf{x})) (h^*(\mathbf{x}) - y) \right] \end{aligned}$$

As before, we can show the last of these three terms is zero:

$$\begin{aligned}
\mathbb{E}_{\mathbf{x}, y} [(\bar{h}(\mathbf{x}) - h^*(\mathbf{x}))(h^*(\mathbf{x}) - y)] &= \mathbb{E}_{\mathbf{x}} \left[\left(\mathbb{E}_{y|\mathbf{x}} [h^*(\mathbf{x}) - y] \right) (\bar{h}(\mathbf{x}) - h^*(\mathbf{x})) \right] \\
&= \mathbb{E}_{\mathbf{x}} \left[\left(h^*(\mathbf{x}) - \mathbb{E}_{y|\mathbf{x}} [y] \right) (\bar{h}(\mathbf{x}) - h^*(\mathbf{x})) \right] \\
&= \mathbb{E}_{\mathbf{x}} [(h^*(\mathbf{x}) - h^*(\mathbf{x}))(\bar{h}(\mathbf{x}) - h^*(\mathbf{x}))] \\
&= \mathbb{E}_{\mathbf{x}} [(0)(\bar{h}(\mathbf{x}) - h^*(\mathbf{x}))] \\
&= 0
\end{aligned}$$

Bias-variance decomposition. Substituting the remaining two terms back in, we obtain the following decomposition for the expected error for the algorithm $\mathcal{A}$:

$$\mathbb{E}_{\mathbf{x}, y, D} [(h_D(\mathbf{x}) - y)^2] = \underbrace{\mathbb{E}_{\mathbf{x}, D} [(h_D(\mathbf{x}) - \bar{h}(\mathbf{x}))^2]}_{\text{Variance}} + \underbrace{\mathbb{E}_{\mathbf{x}, y} [(h^*(\mathbf{x}) - y)^2]}_{\text{Noise}} + \underbrace{\mathbb{E}_{\mathbf{x}} [(\bar{h}(\mathbf{x}) - h^*(\mathbf{x}))^2]}_{\text{Bias}^2}$$

Intuitive Interpretation: The bias-variance decomposition tells us that the total expected error can be split into three distinct components:

- **Variance:** How much does our model change when trained on different datasets?
- **Bias:** How far is our average model from the optimal predictor?
- **Noise:** The irreducible error inherent in the problem

1.3 Understanding the Components

Let's examine each component of the bias-variance decomposition in detail:

Variance captures how much a model h_D can vary if it's trained on different datasets: note that it measures the expected (squared) difference between a particular model h_D and the expected model $\bar{h}$! An algorithm $\mathcal{A}$ with low variance is one that is likely to return a model similar to $\bar{h}$ regardless of the specific dataset D you end up with¹. On the other hand, a model with high variance is highly sensitive to the specific dataset D that you have. Thus, even if the "expected" model returned by the algorithm $\mathcal{A}$ would achieve good performance, it is highly likely that you end up with a model very different from $\bar{h}$. Intuitively, we might link increasing variance to increasing model complexity: because a more complex model may exactly fit a specific dataset D , the model will look very different from dataset to dataset.

Bias measures the inherent error in your model even if you had access to the average model $\bar{h}$ averaged over all possible datasets. In the setting with high bias, even with an infinite amount of data and access to the expected regressor $\bar{h}$ averaged over all possible datasets, your predictor $\bar{h}(\mathbf{x})$

¹ Assuming of course that $D \sim \mathcal{P}^n$

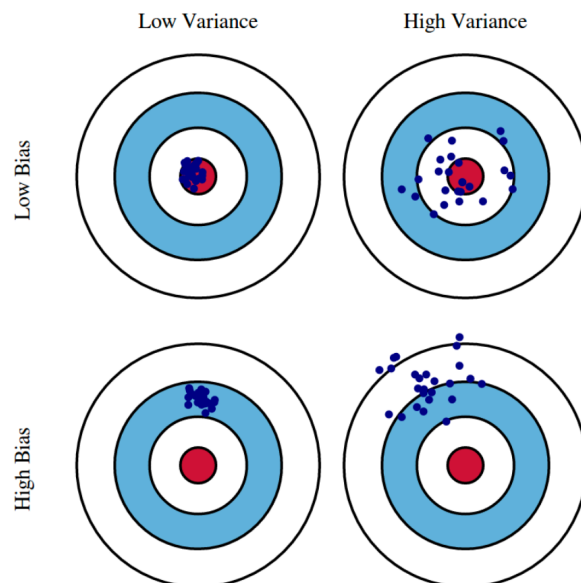


Figure 1: Think of the bullseye as the optimal regressor $h^*(\mathbf{x})$ and the dots on the target board as models returned by your algorithm $\mathcal{A}$. Bias moves the expected regressor $\bar{h}(\mathbf{x})$ away from the bullseye, while variance makes it less likely to hit. Source: <http://scott.fortmann-roe.com/docs/BiasVariance.html>

still returns very different predictions from the optimal regressor $h^*(\mathbf{x})$. In the setting with low bias, if you have enough data that you might achieve a model near $\bar{h}$, then $\bar{h}(\mathbf{x})$ has low difference with the optimal regressor $h^*(\mathbf{x})$. Intuitively, we might link increasing bias to decreasing model complexity: even with an infinite amount of data, it may be hard to achieve good generalization error with a linear model.

Noise is the error you would be left with even if you had access to the true, optimal regressor $h^*(\mathbf{x})$. Note that this error is not "irreducible" per se, as adding more features $\mathbf{x}$ might actually strengthen the optimal regressor $h^*(\mathbf{x})$. However, this error is notably independent of the machine learning algorithm $\mathcal{A}$ that you pick: even if you pick an amazing algorithm that always returns $\bar{h}(\mathbf{x})$ and for which $\bar{h}(\mathbf{x})$ is very close to the optimal regressor, you are still left with noise.

2 Model Debugging

The bias-variance decomposition splits the expected error of an algorithm $\mathcal{A}$ into three distinct components: bias, variance, and noise. This gives us a powerful framework for understanding why our models make errors and how to improve them.

Disclaimer: *While the bias-variance framework provides valuable theoretical insights, it's important to note that these principles don't always translate perfectly to practice. Modern machine learning, especially deep learning, often defies these classical intuitions (as we'll see in the appendix on double descent). The recommendations below should be viewed as useful starting points rather than rigid rules, and may require adaptation based on your specific problem and model.*

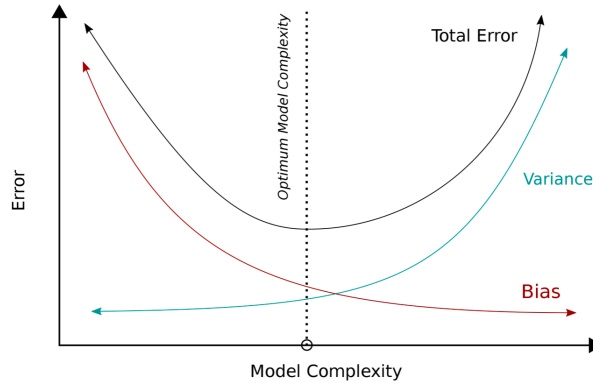


Figure 2: Model complexity and its relation to the two sources of generalization error that depend on your machine learning algorithm, bias and variance. Total error is the sum of bias and variance, plus a noise term that is independent of model complexity. Source: <http://scott.fortmann-roe.com/docs/BiasVariance.html>

2.1 Diagnosing High Variance and Reducing It

Above, we said intuitively that the high variance setting occurs when we have high model complexity that is making us very sensitive to the particular dataset that we have. We know with high variance our generalization error will be high, but what about our training error? In this setting, we have a very complex model and a small enough dataset that our algorithm is sensitive to it. Therefore, it's very likely that we can fit the training dataset well.

The Problem of Overfitting: The dominant symptom of high variance is that the training error is much lower than the test error. This is a classic sign of **overfitting** - your model has learned to memorize the training data rather than generalize from it. Your model is capturing noise in the training data rather than the underlying pattern.

To combat high variance, we have several options:

1. **Add more training data** - This makes the model less sensitive to any particular dataset D .
2. **Reduce model complexity** - Simplify your model to make it less able to fit noise in the data.
3. **Increase regularization** - Add constraints to your model to prevent overfitting. For linear models, you can apply ℓ_1 or ℓ_2 regularization techniques that penalize large coefficient values. When working with RBF kernels, increasing the σ parameter (where $K(\mathbf{x}, \mathbf{x}') = \exp(-\|\mathbf{x} - \mathbf{x}'\|^2/2\sigma^2)$) makes the kernel smoother and less prone to capturing noise. For decision trees, limiting their depth prevents them from creating overly specific branches that memorize training examples rather than learning generalizable patterns.
4. **Use early stopping** - Stop training before the model overfits to the training data.

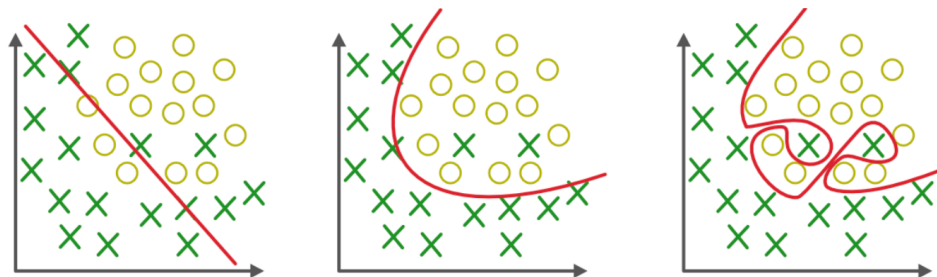


Figure 3: Underfitting and overfitting models. The underfitting model (left) is too simple to capture the data pattern. The overfitting model (right) captures noise rather than the underlying pattern. Source: <https://www.geeksforgeeks.org/underfitting-and-overfitting-in-machine-learning/>

2.2 Diagnosing High Bias and Reducing It

The high bias setting occurs when we have low model complexity that, even with large amounts of data, results in high error. In the setting where bias is much higher than variance, we would expect that our error is dominated by the discrepancy between the average regressor $\bar{h}(\mathbf{x})$ and the optimal one $h^*(\mathbf{x})$. Since our error is not dominated by variation across different datasets (e.g., $h_S(\mathbf{x})$ is usually similar to $\bar{h}(\mathbf{x})$), we might expect the error to be similar on both the training set and the test set.

The Problem of Underfitting: The dominant symptom of high bias is that the training error is about the same as the test error, but both are too high. Your model is simply not powerful enough to capture the underlying patterns in the data.

To combat high bias, we have several options:

1. **Add more features** if possible - This gives your model more information to work with and can reduce both bias and noise:
 - Reduces bias by allowing the model to capture more complex relationships
 - Reduces noise by providing additional predictive signals that help distinguish true patterns from random variations
2. **Increase model complexity** - Use a more sophisticated model class that can capture more complex patterns. This includes moving from linear to nonlinear models, using higher-degree polynomial features, and increasing tree depth for decision trees.
3. **Decrease regularization** - Relax the constraints on your model.

A Modern Perspectives on Bias-Variance (Optional Reading)

While the classical bias-variance decomposition remains foundational, recent research has revealed an important phenomenon that challenges our traditional understanding:

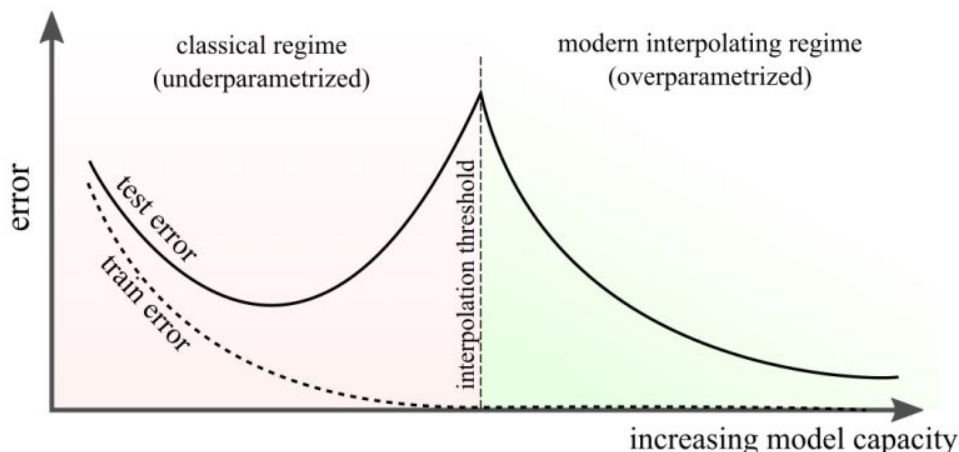


Figure 4: The double descent phenomenon. In the classical regime (left of the interpolation threshold), we observe the familiar U-shaped test error curve. However, in the modern regime (right of the threshold), the test error decreases again as model complexity increases, creating a “double descent” pattern. Meanwhile, training error reaches zero at the interpolation threshold and remains there. Based on [1].

Double Descent. The classical bias-variance tradeoff suggests a U-shaped risk curve (as shown in Figure 2), where increasing model complexity eventually leads to higher error due to variance. However, modern research has discovered that when model complexity increases beyond the interpolation threshold (where training error reaches zero), test error often *decreases again* rather than continuing to increase. This creates a “double descent” curve rather than the classical U-shape. Furthermore, the number of parameters alone is often not the right metric for model complexity—models with millions of parameters can sometimes generalize better than smaller models.

This phenomenon challenges the conventional wisdom that models complex enough to perfectly fit training data will necessarily generalize poorly. It suggests that our understanding of what constitutes “too complex” may need revision, especially in the context of modern deep learning models.

One explanation for this surprising behavior is *implicit regularization*. When a model is overparam-eterized, infinitely many parameter configurations can achieve zero training error, but optimization algorithms like stochastic gradient descent tend to converge to solutions with better generalization properties. These algorithms exhibit an implicit bias toward “simpler” functions, even without explicit regularization terms. This phenomenon helps explain why increasing model capacity beyond what’s needed to fit training data can actually improve generalization rather than harm it—directly contradicting classical intuitions about overfitting.

References

- [1] Belkin, M., Hsu, D., Ma, S., & Mandal, S. (2019). Reconciling modern machine-learning practice and the classical bias-variance trade-off. *Proceedings of the National Academy of Sciences*, 116(32), 15849-15854.